

Etude technique

5^{ème} année d'Ecole d'Ingénieurs

2024/2025

Dispositif de lutte contre les frelons asiatiques – Système de pointage de la cible



Etudiants : COSSON Lucas et GRIMAL Hugo

Encadrant : OUSSET Michel

Spécialité : Mécatronique

REMERCIEMENT

Nous tenons, Lucas COSSON et Hugo GRIMAL, à exprimer nos sincères remerciements à toutes les personnes qui ont contribué de près ou de loin au bon déroulement du début de notre projet.

Tout d'abord, nous souhaitons remercier l'école ENSIL-ENSCI pour avoir mis à notre disposition leurs locaux tout au long de cette année, ainsi que pour leur confiance en nous prêtant le matériel dont nous avons besoin.

Nous tenons également à remercier chaleureusement notre tuteur de projet, Monsieur Michel OUSSET, pour son accompagnement tout au long de ce semestre. Son expertise, son expérience et son investissement ont été d'une grande aide dans l'encadrement et le lancement de ce projet.

Merci à tous pour votre aide précieuse et votre engagement à nos côtés.

Lucas COSSON, Hugo GRIMAL



Table des matières

Introduction	4
1. Détection des frelons par Intelligence Artificielle	5
A. Contexte / Cahier des charges	5
B. Utilisation de l'Intelligence Artificielle : YOLOv8	5
B.1. Entraînement du modèle	6
B.2. Utilisation	6
B.2. Utilisation du modèle / Résultat.....	8
2. Système de pointage des frelons.....	10
A. Fonctionnement du système	10
C. Montage du système	15
D. Pilotage du système	18
3. Combinaison de l'intelligence Artificielle et le système de pointage.....	20
A – Communication Python / Arduino UNO	20
4. Implémentation sur GPU	23
Bibliographie	26
ANNEXES	27
A – Code Python.....	27
B – Code Arduino.....	30
C – Architecture Code Arduino	33
5. Développement durable / éthique/ santé et sécurité au travail	34
A - L'intégration de la thématique dans le Développement Durable.....	34
B - L'intégration de la thématique dans la sécurité	34
Résumé	35



Introduction

Le frelon asiatique est une espèce invasive originaire d'Asie, dont la présence en Europe menace les populations d'abeilles domestiques. Le frelon s'attaque aux ruches, mettant en danger la pollinisation et, l'équilibre des écosystèmes ainsi que la production apicole. Contrairement aux abeilles asiatiques, qui ont développé une stratégie de défense collective en surchauffant le frelon jusqu'à sa neutralisation, les abeilles européennes sont largement démunies face à cette menace.

Ce projet vise à développer un système autonome permettant de détecter, suivre et neutraliser les frelons asiatiques à l'aide d'une intelligence artificielle basée sur YOLOv8, de moteurs pour la poursuite de la cible et d'un laser pour la dissuasion ou l'élimination du frelon. L'objectif est de protéger les ruches tout en minimisant les impacts sur l'environnement et les autres espèces.

En amont de quelconques recherches ou travaux, il est essentiel de structurer les étapes clés du projet afin de rester concentré et d'optimiser le temps de développement. Nous avons donc débuté par deux axes de travail menés en parallèle : d'une part, la mise en place d'un système de détection des frelons asiatiques basé sur l'intelligence artificielle YOLOv8, et d'autre part, la réflexion sur un mécanisme de pointage permettant de suivre et viser le plus précisément possible la cible, le frelon.

Une fois ces bases établies, nous avons poursuivi avec la conception du système, incluant la modélisation et la sélection des composants mécaniques et électroniques nécessaires. Ensuite, nous avons réalisé des tests sur la détection et le suivi, avant d'intégrer l'ensemble dans un prototype fonctionnel. Puis pour finir nous avons exploré une piste de travail avec une carte de développement possédant un GPU de Nvidia.



1. Détection des frelons par Intelligence Artificielle

A. Contexte / Cahier des charges

Tout d'abord pour pouvoir détecter efficacement le frelon, il faut tout d'abord comprendre son fonctionnement pour pouvoir comprendre comment le frelon chasse les abeilles. Les frelons volent en stationnaire devant l'entrée de la ruche et attendent le bon même moment pour attraper leurs proies.



Figure 1 - Frelon en vol stationnaire devant l'amas d'abeilles

Comme on peut l'observer sur la figure 1, le frelon, entouré en rouge, vole en stationnaire devant l'amas d'abeilles, attendant le moment opportun pour capturer sa proie.

Cette étude nous a permis d'établir un cahier des charges simple pour la détection du frelon. Notre procédure doit être capable d'identifier un frelon d'une taille comprise entre 2 et 3 cm, en tenant compte du fait qu'une caméra peut être positionnée à une hauteur de 80 cm à 1 m, filmant vers le sol, une zone d'un mètre par un mètre. Le choix de la caméra n'a pas été traité dans ce projet, car nous utilisons des vidéos existantes pour la programmation.

B. Utilisation de l'Intelligence Artificielle : YOLOv8

Aucun des deux étudiants n'avait travaillé sur l'intelligence artificielle auparavant, nous avons donc dû apprendre en autodidacte.

Nos recherches nous ont rapidement orientés vers l'intelligence artificielle YOLOv8, développée par l'entreprise Ultralytics, spécialisée dans le traitement d'images et de vidéos. Le modèle YOLOv8 que nous avons téléchargé était capable de détecter des personnes, des voitures, des téléphones, de la nourriture et d'autres éléments du quotidien.

En revanche, le frelon n'était pas détecté par le modèle, nous avons donc dû l'entraîner spécifiquement pour cette tâche.



B.1. Entraînement du modèle

En nous appuyant sur la documentation Ultralytics disponible en ligne (voir bibliographie), nous avons découvert qu'il était possible d'entraîner le modèle selon nos besoins.

La première étape essentielle pour l'entraînement du modèle consistait à collecter un maximum de photos afin de l'alimenter en données. Durant les premières semaines, nous avons utilisé des photos trouvées sur Internet.



Figure 2 - Images de frelon asiatique disponible sur internet

Ces photos n'étaient clairement pas suffisantes, car elles ne correspondaient pas à la réalité des conditions en situation réelle. Il était donc nécessaire de trouver une nouvelle banque d'images.

Monsieur OUSSET nous a alors fourni un grand nombre de ses photos et vidéos personnelles, que nous avons pu utiliser pour entraîner notre modèle. Ces nouvelles images correspondaient parfaitement à ce que notre système devait apprendre à reconnaître.



Figure 3 - Photos de M. OUSSET



Une fois les photos triées et les captures d'écran des vidéos réalisées, nous avons recensé une petite centaine d'images exploitables pour entraîner le modèle.

La seconde étape de l'entraînement consistait à repérer manuellement le frelon sur chaque photo. Pour cela, nous avons utilisé le logiciel gratuit LabelImg.exe. Sur chaque image, il fallait délimiter le frelon en créant une boîte autour de lui. Le logiciel générait alors un fichier texte contenant les coordonnées de la boîte créée.



Figure 4 - Capture d'écran du logiciel LABELIMG

Une fois la photo importée, il suffit de se mettre en mode « YOLO », cadre bleu à gauche sur la figure 4, et de créer une boîte autour du frelon. La boîte créée est difficilement visible, comme indiqué dans le cadre rouge sur la figure 4. Une fois la boîte dessinée, on attribue la classe « frelon ». Cette opération est répétée une centaine de fois afin d'obtenir autant de fichiers texte que d'images.

Nous pouvons maintenant entraîner notre modèle en utilisant la base de données que nous venons de créer. Pour cela, l'entraînement se fait sous Python en exécutant un court programme. Le fichier config.yaml est le fichier qui permet de récupérer les fichiers texte et les images. Nous avons deux paramètres à changer les epochs et la taille des images. Le paramètre le plus important à modifier est le premier : epochs, qui correspond au nombre de fois où le modèle va s'entraîner sur chaque image.

```
1 from ultralytics import YOLO
2
3 #Charger le modèle YOLO
4 model = YOLO ("yolov8n.pt" )
5
6 #Entraîner le modèle
7 model.train(data=r'C:\Users\Lucas\Desktop\PROJET_FRELON\Premier_prog\config.yaml', epochs=750, imgsz=640)
```

Figure 5 - Programme d'entraînement du modèle

Nous avons bien évidemment réalisé plusieurs tests. Le dernier en date a été effectué avec 750 itérations. L'entraînement a duré toute une nuit, notamment en raison de l'ordinateur que nous avons utilisé, qui n'est pas forcément optimal pour l'intelligence artificielle, celle-ci nécessitant une grande puissance de calcul.



À l'issue de ce processus, le programme génère un fichier .pt, correspondant au modèle entraîné et prêt à être utilisé.

B.2. Utilisation du modèle / Résultat

La partie la plus fastidieuse a été réalisée, rendant l'utilisation beaucoup plus simple. Il suffit, dans un programme Python, de choisir le modèle que nous voulons utiliser et de sélectionner un média sur lequel l'appliquer.

Nous avons ajouté une boîte englobante autour de la détection, ainsi que l'affichage du centre de cette boîte avec ses coordonnées en pixels, comme illustré dans le code ci-dessous :

```
# Afficher le rectangle sur l'image
cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
label = f"{model.names[int(class_id)]}: {confidence:.2f}"
cv2.putText(frame, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

# Calculer les coordonnées du centre de la boîte
center_x = (x1 + x2) // 2
center_y = (y1 + y2) // 2
cv2.circle(frame, (center_x, center_y), radius=5, color=(0, 0, 255), thickness=-1)

# Afficher les coordonnées du centre sur l'image
center_text = f"Centre: ({center_x}, {center_y})"
cv2.putText(frame, center_text, (center_x, center_y), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)
```

Ce couple de coordonnées nous sera extrêmement utile pour le pointage, comme nous le verrons par la suite. Nous affichons également l'indice de confiance de l'algorithme, qui indique le degré de certitude du modèle quant à la détection d'un frelon. Cet indice est compris entre 0 et 1. Le code réalisant toutes ces opérations est disponible en annexe A. La Figure 6 présente différentes images extraites d'une vidéo montrant un frelon en mouvement.





Figure 6 - Captures d'écran de la détection



2. Système de pointage des frelons

A. Fonctionnement du système

Une fois la détection des frelons effectuée, nous devons concevoir un système de pointage pour un laser, qui nous permettra de neutraliser les frelons.

Dans les années passées, d'autres étudiants avaient tenté de réaliser un système de pointage du laser en utilisant des miroirs inclinés par des moteurs. Lors de notre première réunion de projet, nous avons immédiatement envisagé une approche différente, en nous inspirant des tourelles de défense utilisées dans le secteur militaire.



Figure 7 - Tourelles de défense

Ce système nous a paru plus simple à implémenter, plus rapide et plus précis. Nous avons donc décidé d'intégrer deux moteurs dans notre système : l'un permettant un mouvement sur l'axe horizontal et l'autre sur l'axe vertical. Ces deux moteurs sont montés l'un sur l'autre afin d'assurer une rotation simultanée sur les deux axes. Le laser, quant à lui, est fixé sur l'assemblage de ces deux moteurs pour suivre précisément la cible.

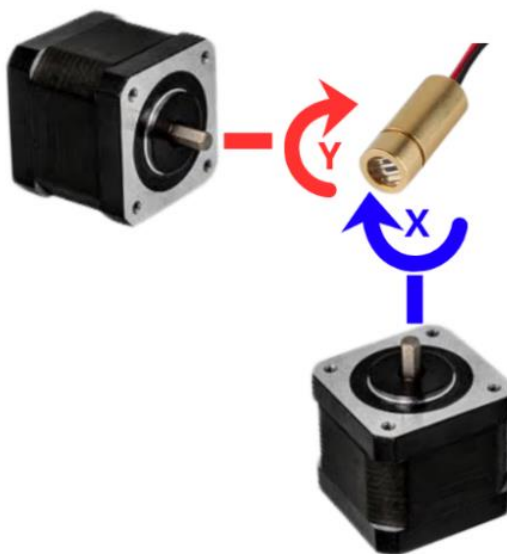


Figure 8 - Schéma alignement moteur



Après plusieurs réflexions, nous avons décidé de placer la source du laser exactement sur l'axe de rotation de chaque moteur. Cela nous permet d'avoir un contrôle précis du pointage. Si cette condition n'est pas respectée, un décalage se produirait lors du ciblage du laser, en raison d'un effet de bras de levier créé par la distance entre l'axe du moteur et la source du laser.

Cette condition est donc essentielle pour garantir une précision optimale. En effet, un frelon asiatique mesurant environ 2 cm, toute erreur de pointage due à ce décalage risquerait de faire manquer la cible.

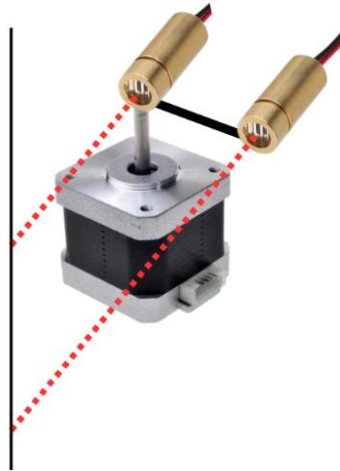


Figure 9 - Schéma décalage bras de levier

B. Les composants utilisés

Pour concevoir notre système, nous avons sélectionné différents composants en fonction des exigences définies dans notre cahier des charges.

Comme mentionné précédemment, une précision élevée est indispensable pour pointer un frelon avec le laser, tout comme une grande rapidité de pointage. C'est pourquoi nous avons choisi d'utiliser des moteurs pas à pas. Ces moteurs répondent parfaitement à nos besoins, car ils offrent une excellente précision, une bonne vitesse de réponse et un couple suffisant pour assurer un contrôle optimal du laser. Nous avons utilisé les références suivantes : 17HS15-0404S



Figure 10 - Moteur Pas à Pas



Nos moteurs pas à pas ont une résolution de 200 pas par tour, soit $1,8^\circ$ par pas. Bien que cette précision puisse sembler suffisante, elle devient problématique dans notre cas. En effet, le frelon n'est pas positionné directement à la source du laser, mais à une distance de 800 mm de notre système, soit la distance que nous avons fixée entre la ruche et le dispositif.

Cette configuration engendre un effet de bras de levier, amplifiant le déplacement du point d'impact du laser. Comme illustré sur la figure ci-dessous, une rotation de $1,8^\circ$ du moteur entraîne un décalage d'environ 27 mm à 800 mm de distance. Ce manque de précision est critique, car un frelon mesure lui-même environ 20 mm. Ainsi, à chaque pas du moteur, nous risquons de manquer notre cible.

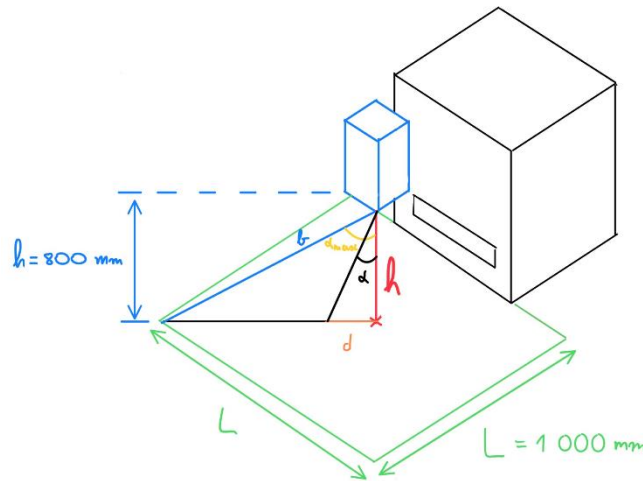
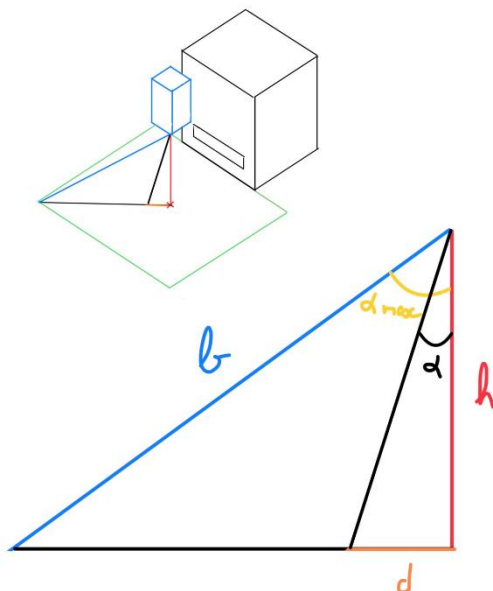


Figure 11 - Schéma représentatif de la situation



$$b = \sqrt{800^2 + \left(\frac{\sqrt{2}}{2} \times 500\right)^2} = 874 \text{ mm}$$

$$\alpha_{max} = \cos^{-1}\left(\frac{h}{b}\right) \times \frac{360}{2\pi} = 23,74^\circ$$

$$\alpha = 1,8^\circ$$

$$d = h \times \tan(\alpha) = 27,20 \text{ mm}$$



Pour pallier ce problème, nous avons décidé d'utiliser une carte de contrôle pour nos moteurs pas à pas, permettant d'activer le micro-stepping. Cette carte se connecte directement à la carte Arduino, facilitant ainsi son intégration et son pilotage.

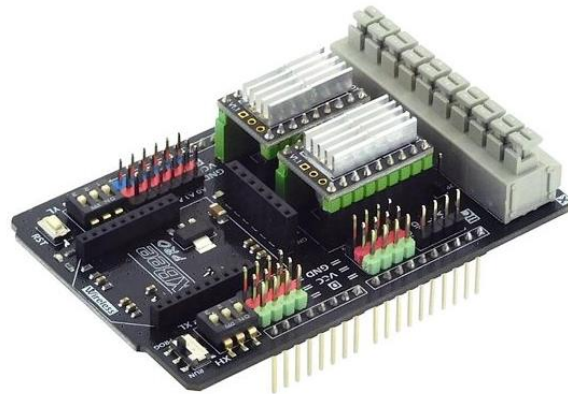


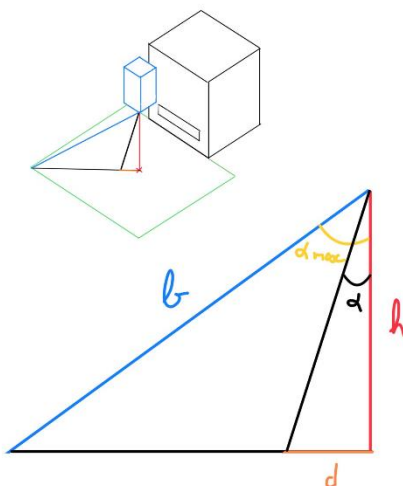
Figure 12 - Carte de contrôle micro-stepping

Cette technique consiste à diviser chaque pas du moteur en plusieurs micro-pas, augmentant ainsi considérablement la précision du déplacement.

Notre carte de contrôle prend en charge plusieurs niveaux de micro-stepping : 1/8, 1/16 et 1/32. Nous avons opté pour la plus haute résolution, soit 1/32, ce qui signifie que chaque pas de $1,8^\circ$ est divisé en 32 micro-pas. Ainsi, la résolution angulaire passe de $1,8^\circ$ à :

$$\frac{1,8^\circ}{32} = 0,0563^\circ$$

Comme illustré dans la figure ci-dessous, cette nouvelle précision compense considérablement l'effet de bras de levier. À une distance de 800 mm, une rotation de $0,0563^\circ$ entraîne un décalage d'environ 0,85 mm au point d'impact du laser. Cette précision est largement suffisante pour cibler un frelon de 20 mm avec une grande fiabilité.



$$b = 866 \text{ mm}$$

$$\alpha_{max} = 22^\circ$$

$$\alpha = \frac{1,8^\circ}{32} = 0,05625^\circ$$

$$d = h \times \tan(\alpha) = 0,85 \text{ mm}$$



Afin de piloter notre système de pointage, nous avons utilisé une carte Arduino Uno. Cette carte, simple d'utilisation et largement documentée, nous permet de contrôler efficacement la carte de gestion des moteurs pas à pas, tout en facilitant l'implémentation et le développement de notre système.



Figure 13 - Carte Arduino Uno

Nous avons également intégré deux capteurs de fin de course, un pour chaque moteur. Nous nous sommes rapidement rendu compte que notre système nécessitait une initialisation très précise au démarrage afin d'assurer un pointage fiable.

Pour répondre à ce besoin, nous avons conçu et modélisé des supports en impression 3D permettant d'activer les capteurs lors du passage du moteur. Ces capteurs nous permettent d'établir une position d'origine, garantissant que le système démarre toujours du même point. Cela facilite grandement le calibrage et le contrôle du pointage dans la suite du fonctionnement du système.

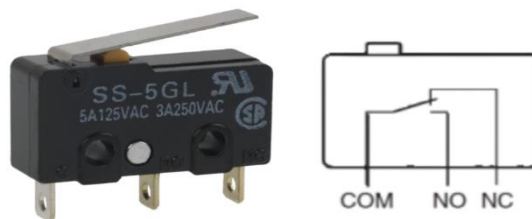


Figure 14 - Capteur de fin de course

Pour réaliser notre maquette à échelle réduite, nous avons utilisé également un laser servant uniquement au pointage lumineux. Ce laser n'était pas suffisamment puissant pour avoir un effet sur un frelon, mais il nous a permis de visualiser précisément le point d'impact du futur laser de neutralisation. Nous l'avons alimenté en 3,3V, ce qui était suffisant pour nos tests de repérage et d'alignement du système.



Figure 15 - Laser



Pour finir, nous avons également intégré un relais pilotable à notre système. Ce composant nous permet de contrôler l'alimentation du laser afin qu'il ne reste pas allumé en permanence. Sans ce relais, le laser fonctionnerait en continu, même en l'absence de frelon, ce qui serait inutile et énergivore. Grâce à cette solution, le laser ne s'allume que lorsqu'un frelon est détecté et correctement pointé, optimisant ainsi son utilisation et l'efficacité du système.



Figure 16 - Relais

C. Montage du système

Une fois tous les composants de notre système sélectionnés, nous avons pu passer à la conception en 3D.

Dans un premier temps, nous avons conçu un support pour le moteur, permettant de le fixer à la verticale afin de réaliser notre axe X. Cette pièce assure une fixation stable et précise du moteur, garantissant un mouvement fluide et fiable pour le pointage du laser.



Figure 17 - Support Moteur axe X

Par la suite, nous avons conçu une pièce se fixant sur le premier moteur à l'aide d'inserts, permettant d'ajouter une vis de pression pour bloquer la rotation et ainsi solidariser le support avec le moteur. Cette pièce assure la rotation du second moteur autour de l'axe X. Ce nouveau moteur permet ensuite d'obtenir le mouvement sur l'axe Y, complétant ainsi notre système de pointage en deux axes pour un contrôle précis du laser.





Figure 18 - Support Moteur axe Y

Comme on peut le voir sur la photo ci-dessus, nous avons conçu un déport pour le moteur assurant la rotation sur l'axe Y. Cette conception permet, comme expliqué précédemment, d'aligner précisément la source du laser avec les axes de rotation des deux moteurs. Cet alignement est essentiel pour éviter tout effet de bras de levier et garantir un pointage précis du laser sur la cible.

Une fois les deux supports moteurs réalisés, nous avons ajouté les supports des capteurs de fin de course. Ces supports ont été conçus comme des pièces séparées plutôt que d'être imprimés directement avec les supports moteurs. Cette approche nous permet, en cas d'ajustement nécessaire, de modifier uniquement les supports des capteurs sans devoir réimprimer l'ensemble du système.

Ces supports sont équipés d'ergots assurant une fixation solide des capteurs de fin de course. Une fois assemblés, ils sont fixés sur les supports moteurs, comme illustré sur la photo ci-dessous.



Figure 19 - Support Capteur de fin de course

Il ne reste plus qu'à concevoir le support du laser. Comme mentionné précédemment, il est essentiel que la source du laser soit parfaitement alignée avec les axes de rotation des deux moteurs.

De plus, ce support doit également actionner le capteur de fin de course lors de la rotation. Pour cela, nous avons ajouté un épaulement qui vient en contact avec le capteur. Nous avons également intégré une vis de pression afin d'assurer une fixation solide du laser sur son support, garantissant ainsi sa fixation. Nous avons directement encastré le laser dans ce support.



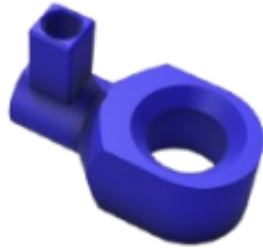
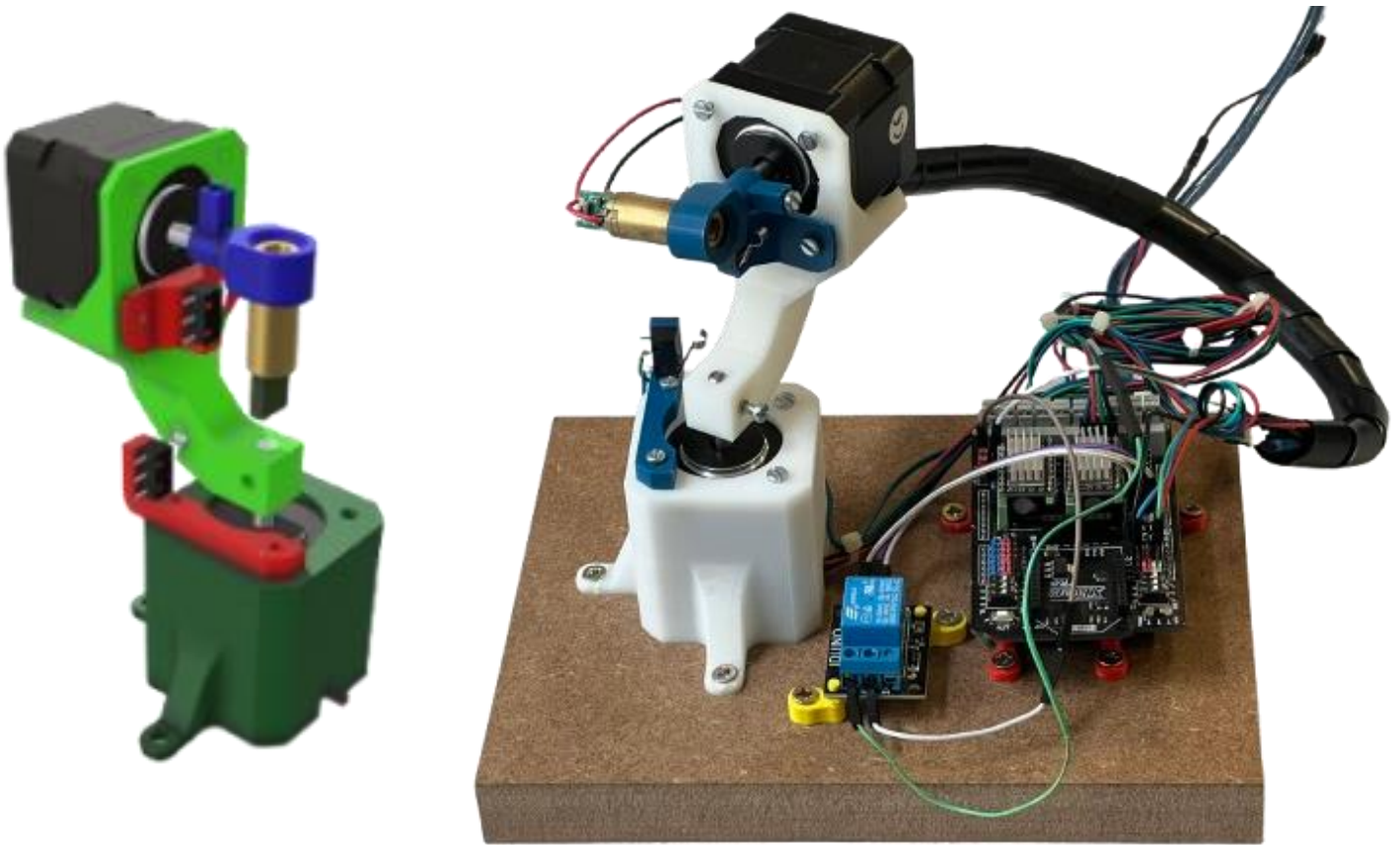


Figure 20 - Support Laser

Une fois la modélisation de tous les éléments terminés, nous les avons imprimés en 3D, puis assemblés. Afin d'assurer la stabilité de notre système, nous avons fixé l'ensemble sur une planche en bois. Cette base rigide permet de limiter les vibrations et d'assurer un fonctionnement précis du mécanisme.



D. Pilotage du système

Une fois l'assemblage de toutes les pièces et tous les composants reliés, nous pouvons maintenant passer à la programmation de notre système. Cette partie concerne le code Arduino, qui est détaillé dans l'annexe B. L'architecture du code est en annexe C

En résumé, le programme permet de piloter les deux moteurs pas à pas. Lorsqu'il reçoit des coordonnées, l'Arduino gère les déplacements des moteurs tout en prenant en compte les limites de déplacement, grâce aux capteurs de fin de course. De plus, le programme assure la gestion du laser, en activant ou désactivant ce dernier à l'aide d'un relais.

Dans la première partie du code, nous initialisons chaque pin ainsi que toutes les variables nécessaires au bon fonctionnement du système. Par exemple, nous définissons les vitesses des moteurs. Nous avons également déclaré des variables pour les limites d'axes, ce qui nous permet de définir une zone de travail et ainsi éviter que le système dépasse ces limites et heurte les capteurs de fin de course.

```
int M1dirpin = 7; // Broche direction moteur X
int M1steppin = 6; // Broche pas moteur X
int M1en = 8; // Broche activation moteur X
int M2dirpin = 4; // Broche direction moteur Y
int M2steppin = 5; // Broche pas moteur Y
int M2en = 12; // Broche activation moteur Y

int limitSwitchX = 2; // Capteur de fin de course pour l'axe X
int limitSwitchY = 3; // Capteur de fin de course pour l'axe Y
int relayPin = 9; // Broche pour contrôler le relais (laser)

#define PAS_PAR_TOUR_COMPLET 6400 // Nombre de pas par tour complet du moteur
#define MAX_PAS_X 5300 // Maximum en nombre de pas pour l'axe X
#define MIN_PAS_X 300 // Minimum en nombre de pas pour l'axe X
#define MAX_PAS_Y 2200 // Maximum en nombre de pas pour l'axe Y
#define MIN_PAS_Y 300 // Minimum en nombre de pas pour l'axe Y
```

Une fois l'initialisation effectuée, nous activons les moteurs et, par souci de sécurité, nous éteignons par défaut le relais, ce qui permet d'éviter tout fonctionnement accidentel du laser. Ensuite, nous lançons une initialisation de la position. Cette étape est cruciale pour garantir que le système commence à une position de référence précise, ce qui assure une bonne précision dès le démarrage et permet de repartir à partir de la même origine à chaque mise en route du système.

```
digitalWrite(M1en, LOW); // Activation moteur X
digitalWrite(M2en, LOW); // Activation moteur Y
digitalWrite(relayPin, LOW); // Par défaut, relais (et laser) éteints
initialiserPosition(); // Initialisation et retour des moteurs à (0,0)
```

Une fois l'initialisation de la position effectuée, le système est prêt à être utilisé. Il attend alors de recevoir une communication série, préalablement ouverte, et une commande au format X, Y, état du laser, où X et Y correspondent aux coordonnées en nombre de pas et où l'état du laser est défini par 1 pour l'allumer et 0 pour l'éteindre.

```
if (Serial.available() > 0) {
    lastTime = millis(); // Mise à jour du temps de dernière commande reçue
    input = Serial.readStringUntil('\n'); // Lire la ligne entrée
    input.trim(); // Supprimer les espaces blancs
    Serial.print("Commande reçue : ");
    Serial.println(input);
}
```



Les valeurs des coordonnées sont bornées entre des limites définies afin d'éviter toute sortie de la zone de travail. Si une commande dépasse ces limites, la valeur est automatiquement ajustée au maximum ou au minimum autorisé, garantissant ainsi un fonctionnement optimal et sécurisé du système.

```
// Limiter les coordonnées pour ne pas dépasser les pas minimaux et maximaux
cibleX = constrain(cibleX, MIN_PAS_X, MAX_PAS_X);
cibleY = constrain(cibleY, MIN_PAS_Y, MAX_PAS_Y);
laserState = constrain(laserState, 0, 1); // État du laser doit être 0 ou 1
```

Une fois les coordonnées reçues en nombre de pas, notre programme calcule le nombre de micro-pas nécessaires pour atteindre la cible. Il actionne ensuite les moteurs pas à pas en envoyant les impulsions requises jusqu'à ce que la position souhaitée soit atteinte. Ce processus garantit un déplacement précis du système de pointage vers la cible définie.

```
//Déplacer les moteurs vers les nouvelles coordonnées avec la vitesse normale
deplacerAuxCoordonnees(cibleX, cibleY, VITESSE_NORMALE);
```

Une fois que les moteurs ont atteint la position cible, le programme met à jour la position actuelle avec les nouvelles coordonnées. Si aucune nouvelle commande n'est reçue dans un délai de cinq secondes, le laser s'éteindra automatiquement afin d'éviter tout fonctionnement inutile

```
// Si rien n'a été reçu depuis un certain temps, éteindre le laser
if (millis() - lastTime > TIMEOUT) {
    digitalWrite(relayPin, LOW); // Éteindre le laser
    Serial.println("Aucune commande reçue, laser éteint.");
}
```



3. Combinaison de l'intelligence Artificielle et le système de pointage

A – Communication Python / Arduino UNO

Dans cette partie, nous étudions l'interaction et la communication entre nos deux programmes. Cette section fait référence à l'annexe A. Afin de pouvoir transmettre les résultats de la détection du frelon et piloter le laser, nous avons utilisé la communication série USB de l'Arduino, branché directement sur le PC.

Le rôle de l'Arduino est uniquement de recevoir les coordonnées envoyées et d'activer le laser en conséquence. Il oriente ainsi le système vers la cible et allume le laser si nécessaire. En revanche, le programme Python exécuté sur l'ordinateur joue le rôle de "cerveau" : il détecte le frelon, détermine ses coordonnées, puis les transmet à l'Arduino pour exécution.

```
7  # Initialiser la connexion série avec l'Arduino
8  serial_connection = serial.Serial('COM8', 9600)
9  time.sleep(2) # Temps pour établir la connexion série
```

C'est ici que les coordonnées de la boîte englobante du frelon nous sont extrêmement utiles. Comme mentionné précédemment, pour piloter les moteurs pas à pas, nous utilisons le nombre de pas à effectuer, tandis que notre code Python de détection fournit des coordonnées en pixels. Il est donc impératif, avant d'envoyer les données à l'Arduino, de convertir les pixels en nombre de pas.

Voici comment la conversion a été effectuée suivant ce principe :

Tout d'abord, nous avons récupéré les dimensions de la vidéo de cette manière.

```
19 # Récupérer les dimensions de la vidéo source
20 video_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)) # Largeur de la vidéo
21 video_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)) # Hauteur de la vidéo
22 print(f"Dimensions de la vidéo source: Largeur = {video_width}, Hauteur = {video_height}")
```

À environ 80cm d'un tableau blanc, nous avons défini une zone de travail dans laquelle nous souhaitons que le laser se déplace. Nous avons fixé la largeur de cette zone à 800 pas, valeur choisie selon notre place disponible sur le tableau. Pour la hauteur, nous avons conservé le même rapport d'aspect que la vidéo source. Nous pouvons ainsi calculer le rapport entre nos deux formats différents, nous obtenons l'échelle pour passer de pixel à des pas Arduino.

```
30 # Dimensions de la zone de travail sur l'Arduino (en pas Arduino)
31 zone_width_arduino = 800 # Largeur de la zone de travail
32 zone_height_arduino = zone_width_arduino * (video_height / video_width) # Hauteur de la zone de travail
33
34 print(f"Zone travail arduino : Largeur = {zone_width_arduino}, Hauteur = {zone_height_arduino}")
35
36 # Calcul des proportions de la vidéo
37 scale_x = zone_width_arduino/video_width
38 scale_y = zone_height_arduino/video_height
```



À partir du nombre de pas minimum en largeur de notre zone de travail (minX) et du nombre de pas maximum en hauteur (maxY), nous définissons le point (minX, maxY) comme étant le coin supérieur gauche de notre zone de travail. Le couple (center_x, center_y) correspond aux coordonnées du frelon dans l'image. Pour obtenir sa position en pas sur l'Arduino, nous ajustons les coordonnées du point (minX, maxY) en ajoutant ou en soustrayant la position du frelon, convertie en nombre de pas de cette manière :

```
new_x = minX + center_x * scale_x  
new_y = maxY - center_y * scale_y
```

C'est ce couple de coordonnées (new_x, new_y) que nous envoyons à l'Arduino.

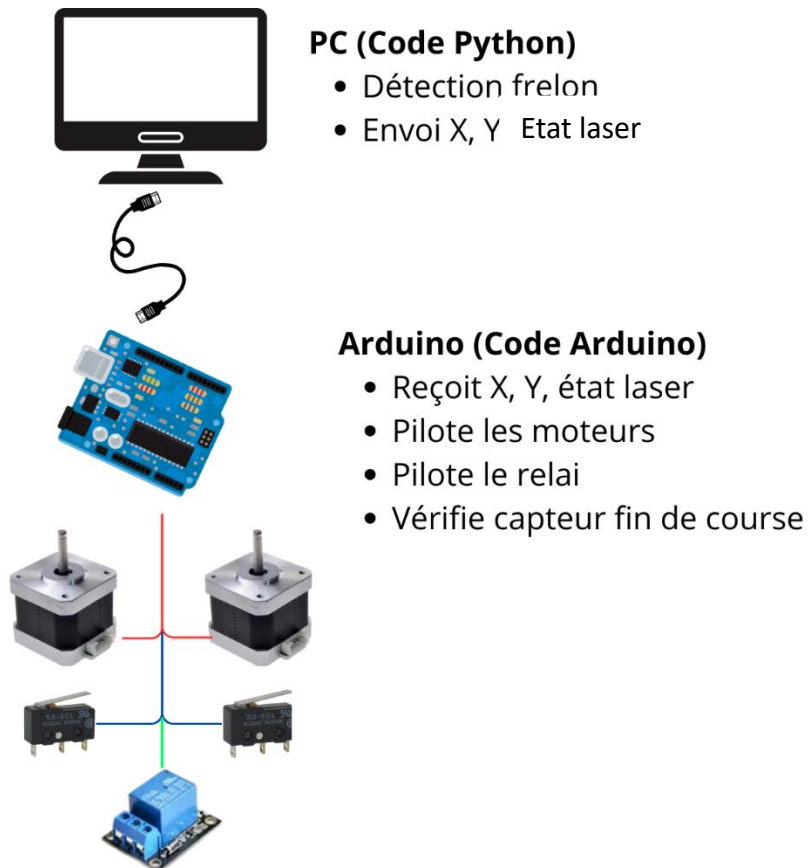
L'ordinateur utilisé est ainsi devenu une pièce maîtresse du projet, détectant les frelons et envoyant les données. Cependant, il s'est révélé insuffisamment puissant, atteignant 100 % de ses capacités lors de l'exécution du programme. C'est pourquoi nous avons décidé d'utiliser un Jetson équipé d'un GPU.

Voici les caractéristiques du PC pas assez puissant :

Système d'exploitation : Windows 11 Famille 64 bits (10.0, build 22631)
Langue : français (Paramètres régionaux : français)
Fabricant du système : Acer
Modèle du système : Aspire A715-75G
BIOS : V1.00
Processeur : Intel(R) Core(TM) i5-9300H CPU @ 2.40GHz (8 CPUs), ~2.4GHz
Mémoire : 8192MB RAM
Fichier de pagination : 16508 Mo utilisé(s), 2941 Mo disponible(s)
Version DirectX : DirectX 12



En résumé, voici comment fonctionne le montage :



4. Implémentation sur GPU

Nous avons essayé de faire fonctionner nos programmes sur un GPU intégré à un Jetson Nano. Nous avons pu récupérer ce Jetson Nano grâce à M. ESCARENO, qui en avait un à configurer. La première étape a donc été de l'initialiser avec l'image flash disponible sur le site de Nvidia. Ne possédant aucune compétence préalable dans ce domaine, nous avons dû apprendre par nous-mêmes.



Figure 21 - Jetson Nano

En ce qui concerne l'installation du Jetson, celle-ci s'apparente à une configuration de Raspberry Pi. Cependant, le Jetson ne disposant pas de puce Wi-Fi intégrée, nous avons dû utiliser une connexion Ethernet pour effectuer les installations nécessaires. Pour cela, nous avons sollicité les techniciens de l'école afin qu'ils nous attribuent une prise Ethernet dédiée à notre projet.

Une fois toutes les configurations et mises à jour effectuées, nous avons commencé à installer les logiciels et bibliothèques nécessaires. Nous avons notamment installé Python ainsi que la bibliothèque Ultralytics, indispensable pour utiliser YOLO. Cette bibliothèque constitue le strict minimum pour exécuter notre code Python et permettre la détection des frelons via l'IA.

Dans un premier temps, nous avons testé notre code initial sans modification. Nous avons rapidement constaté que le Jetson ralentissait encore plus que l'ordinateur utilisé précédemment, alors qu'il devrait être plus performant grâce à la présence d'un CPU et d'un GPU.

Pour mieux comprendre ce problème, nous avons installé "jetson-stats", un outil permettant de surveiller les performances du Jetson. Grâce à cet outil, nous avons remarqué que le CPU était utilisé à 100 %, tandis que le GPU n'était pas exploité du tout. Cela s'explique par le fait que, par défaut, l'exécution du code se fait uniquement sur le CPU.

Ce problème est accentué par le fait que le CPU du Jetson est relativement peu puissant, car il est conçu pour fonctionner en complément du GPU. L'absence d'optimisation du code pour exploiter le GPU explique donc pourquoi nous n'avons observé aucune amélioration des performances. Nous avons donc effectué de nombreuses recherches afin de trouver une solution pour exploiter le GPU dans notre code. Après de multiples efforts pour intégrer cette fonctionnalité, nous avons malheureusement constaté que cela n'était pas possible dans notre cas. En effet, pour utiliser le GPU dans un code Python, il est nécessaire d'avoir « CUDA », un logiciel permettant de programmer les GPU. Cependant, CUDA nécessite Python 3.6, tandis que pour utiliser Ultralytics (et donc YOLO), nous avons besoin de Python



3.8 ou une version ultérieure. Cela crée une incompatibilité, et en conséquence, nous n'avons pas pu utiliser le GPU du Jetson Nano dans notre projet.

Au retour des vacances de Noël, nous avons décidé de commander une nouvelle carte NVIDIA pour pouvoir exploiter pleinement notre système avec les bonnes versions de logiciels. Nous avons alors opté pour le Jetson Orin Nano, une carte 40 fois plus puissante que la précédente et plus récente, ce qui nous permettrait d'installer toutes les dépendances nécessaires à l'utilisation du GPU avec Ultralytics.

Cependant, en mars, nous avons reçu un mail du site où nous avons passé la commande, nous informant que la carte faisait face à des problèmes d'approvisionnement chez le fabricant. En conséquence, le délai de livraison serait d'au moins 10 semaines supplémentaires. À ce jour, nous n'avons donc pas encore reçu la carte. Nous n'avons donc pas pu poursuivre nos recherches ni tester notre système avec un GPU.



Conclusion

Ce projet a permis de concevoir et de tester un dispositif innovant de lutte contre les frelons asiatiques en combinant intelligence artificielle, systèmes mécatroniques et laser. Grâce à l'utilisation du modèle YOLOv8, nous avons pu détecter et suivre en temps réel les frelons, tout en développant un système de pointage précis piloté par des moteurs pas à pas et une carte Arduino.

Les différentes étapes du projet, de l'entraînement du modèle d'IA à l'intégration du Jetson Nano, nous ont confrontés à des défis techniques significatifs. Malgré certaines limitations, notamment liées à la puissance de calcul de l'ordinateur ou du Jetson Nano et à l'indisponibilité du Jetson Orin Nano, nous avons pu valider le concept et démontrer son efficacité en conditions de test.

Les perspectives d'amélioration incluent l'optimisation du modèle de détection pour une meilleure réactivité, et l'intégration d'un laser plus performant.



Bibliographie

- Data Sheet Carte Arduino Uno
<https://www.farnell.com/datasheets/1682209.pdf>
- Documentation Shield moteur pas à pas
https://wiki.dfrobot.com/Stepper_Motor_Shield_For_Arduino_DRV8825_SKU_DRI0023
- Documentation d'aide au démarrage du Jetson Nano
<https://developer.nvidia.com/embedded/learn/get-started-jetson-nano-devkit>
- Documentation Ultralytics
<https://github.com/ultralytics/ultralytics>
- Documentation Cuda
<https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/index.html>



ANNEXES

A – Code Python

```
from ultralytics import YOLO
import cv2
import serial
import time
import keyboard

# Initialiser la connexion série avec l'Arduino
serial_connection = serial.Serial('COM8', 9600)
time.sleep(2) # Temps pour établir la connexion série

# Charger le modèle YOLO
model = YOLO(r"C:\Users\Lucas\Desktop\PROJET_FRELON\Premier_prog\best1.pt")

# Ouvrir la vidéo
video_path = r"C:\Users\Lucas\Desktop\PROJET_FRELON\Premier_prog\video2_2.mp4"
cap = cv2.VideoCapture(video_path)
#cap = cv2.VideoCapture(1, cv2.CAP_DSHOW)

# Récupérer les dimensions de la vidéo source
video_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))/2 # Largeur de la vidéo
video_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))/2 # Hauteur de la vidéo
print(f"Dimensions de la vidéo source: Largeur = {video_width}, Hauteur = {video_height}")

# Coordonnées du centre de la zone de travail (en coordonnées Arduino)
center_x_arduino = 1700 # Exemple de centre en X (en pas Arduino)
center_y_arduino = 850 # Exemple de centre en Y (en pas Arduino)

# Dimensions de la zone de travail sur l'Arduino (en pas Arduino)
zone_width_arduino = 800 # Largeur de la zone de travail
zone_height_arduino = zone_width_arduino * (video_height / video_width) # Hauteur de la zone de travail

print(f"Zone travail arduino : Largeur = {zone_width_arduino}, Hauteur = {zone_height_arduino}")

# Calcul des proportions de la vidéo
scale_x = zone_width_arduino/video_width
scale_y = zone_height_arduino/video_height

print(f"échelle : X = {scale_x}, Y = {scale_y}")

# Calcul des coins de la zone de travail (en coordonnées Arduino)
half_width = zone_width_arduino // 2
half_height = zone_height_arduino // 2

minX = center_x_arduino - half_width
maxX = center_x_arduino + half_width
minY = center_y_arduino - half_height
maxY = center_y_arduino + half_height

# Affichage de la zone de travail calculée
print(f"Zone de travail (en pas Arduino) :")
print(f"Min X: {minX}, Max X: {maxX}")
print(f"Min Y: {minY}, Max Y: {maxY}")

if not cap.isOpened():
    print(f"Erreur : Impossible d'ouvrir la vidéo {video_path}.")
    exit()

# Déplacer le laser vers les 4 coins de la zone de travail
```



```

def move_to_corner(x, y):
    # Envoie des coordonnées au laser
    serial_connection.write(f'{x},{y}\n'.encode())
    print(f'Laser déplacé à X={x}, Y={y}')
    keyboard.wait("space") #Attendre l'espace

# Déplacer le laser vers chaque coin
move_to_corner(minX, minY) # Coin supérieur gauche
move_to_corner(minX, maxY) # Coin inférieur gauche
move_to_corner(maxX, minY) # Coin supérieur droit
move_to_corner(maxX, maxY) # Coin inférieur droit

# Initialisation de l'Arduino
serial_connection.write(f'init\n'.encode())
print(f'Envoyé à l'Arduino : Init')

centert_x=video_width/2
centert_y=video_height/2

newt_x = minX + centert_x * scale_x
newt_y = maxY - centert_y * scale_y
print(f'Centre en arduino test  X_centre: {newt_x}, Y_centre: {newt_y}')

move_to_corner(newt_x,newt_y)

# Une fois les coins visités, commencer la détection
while True:
    ret, frame = cap.read()
    if not ret:
        print("Fin de la vidéo ou erreur de lecture.")
        break

    frame= cv2.resize(frame,(video_width,video_height))
    # Effectuer la détection
    results = model(frame)

    # Parcourir les résultats et afficher les détections
    for result in results:
        boxes = result.bboxes.xyxy # Coordonnées des boîtes
        confidences = result.bboxes.conf # Confiance des détections
        class_ids = result.bboxes.cls # ID des classes

        # Filtrer et afficher seulement si la confiance est > 0.2
        for box, confidence, class_id in zip(boxes, confidences, class_ids):
            if confidence > 0.2:
                x1, y1, x2, y2 = map(int, box)
                cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
                label = f'{model.names[int(class_id)]}: {confidence:.2f}'
                cv2.putText(frame, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

                # Calculer les coordonnées du centre de la boîte
                center_x = (x1 + x2) // 2
                center_y = (y1 + y2) // 2
                cv2.circle(frame, (center_x, center_y), radius=5, color=(0, 0, 255), thickness=-1)

                # Afficher les coordonnées du centre sur l'image
                center_text = f'Centre: ({center_x}, {center_y})'
                cv2.putText(frame, center_text, (center_x, center_y), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)

                new_x = minX + center_x * scale_x
                new_y = maxY - center_y * scale_y

                # Envoie des coordonnées converties à l'Arduino
                serial_connection.write(f'{new_x},{new_y}\n'.encode())
                print(f'Envoyé à l'Arduino : X={new_x}, Y={new_y}')

```



```
cv2.circle(frame, (video_width//2, video_height//2), radius=10 , color=(0 , 0, 255 ), thickness=-1)
# Afficher la frame avec les détections
cv2.imshow("Détection en temps réel", frame)

# Quitter la boucle si 'q' est pressé
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# Libérer la capture et fermer les fenêtres
cap.release()
cv2.destroyAllWindows()
serial_connection.close() # Fermer la connexion série
```



B – Code Arduino

```
int M1dirpin = 7; // Broche direction moteur X
int M1steppin = 6; // Broche pas moteur X
int M1en = 8; // Broche activation moteur X
int M2dirpin = 4; // Broche direction moteur Y
int M2steppin = 5; // Broche pas moteur Y
int M2en = 12; // Broche activation moteur Y

int limitSwitchX = 2; // Capteur de fin de course pour l'axe X
int limitSwitchY = 3; // Capteur de fin de course pour l'axe Y
int relayPin = 9; // Broche pour contrôler le relais (laser)

#define PAS_PAR_TOUR_COMPLET 6400 // Nombre de pas par tour complet du moteur
#define MAX_PAS_X 5300 // Maximum en nombre de pas pour l'axe X
#define MIN_PAS_X 300 // Minimum en nombre de pas pour l'axe X
#define MAX_PAS_Y 2200 // Maximum en nombre de pas pour l'axe Y
#define MIN_PAS_Y 300 // Minimum en nombre de pas pour l'axe Y

#define SLOW_DOWN_PAS 100 // Nombre de pas pour commencer à ralentir

// Définition des vitesses en microsecondes
#define VITESSE_NORMALE 75
#define VITESSE_RALENTIE 200
#define DELAI_BLOCAGE 500 // Délai en millisecondes pour bloquer la position après le déplacement
#define TIMEOUT 5000 // Timeout de 5 secondes sans nouvelle donnée (en millisecondes)

long positionXActuelle = 0;
long positionYActuelle = 0;

void setup() {
  // Initialisation de la communication série
  Serial.begin(9600);
  // Configuration des moteurs et des capteurs de fin de course
  pinMode(M1dirpin, OUTPUT);
  pinMode(M1steppin, OUTPUT);
  pinMode(M1en, OUTPUT);
  pinMode(M2dirpin, OUTPUT);
  pinMode(M2steppin, OUTPUT);
  pinMode(M2en, OUTPUT);
  pinMode(limitSwitchX, INPUT_PULLUP); // Capteur fin de course pour X
  pinMode(limitSwitchY, INPUT_PULLUP); // Capteur fin de course pour Y
  pinMode(relayPin, OUTPUT); // Configurer la broche du relais (laser) comme sortie
  digitalWrite(M1en, LOW); // Activation moteur X
  digitalWrite(M2en, LOW); // Activation moteur Y
  digitalWrite(relayPin, LOW); // Par défaut, relais (et laser) éteints

  initialiserPosition(); // Initialisation et retour des moteurs à (0,0)

  // Message d'invite pour entrer les coordonnées
  Serial.println("Système prêt. Entrez les coordonnées X, Y, et l'état du laser (0 ou 1). Exemple : 1000,500,1");
}

void loop() {
  static unsigned long lastTime = 0; // Temps de la dernière commande reçue
  static String input = ""; // Stocke la commande reçue

  // Vérifier si des données ont été envoyées via la liaison série
  if (Serial.available() > 0) {
    lastTime = millis(); // Mise à jour du temps de dernière commande reçue
    input = Serial.readStringUntil('\n'); // Lire la ligne entrée
    input.trim(); // Supprimer les espaces blancs
    Serial.print("Commande reçue : ");
    Serial.println(input);
  }
}
```



```

// Vérifier si la commande est "init" pour réinitialiser la position
if (input.equalsIgnoreCase("init")) {
    Serial.println("Réinitialisation de la position en cours...");
    initialiserPosition(); // Relancer l'initialisation
    Serial.println("Réinitialisation terminée.");
} else {
    // Séparer les coordonnées X, Y et l'état du laser
    int firstCommaIndex = input.indexOf(',');
    int secondCommaIndex = input.indexOf(',', firstCommaIndex + 1);

    if (firstCommaIndex != -1 && secondCommaIndex != -1) {
        String X_str = input.substring(0, firstCommaIndex);
        String Y_str = input.substring(firstCommaIndex + 1, secondCommaIndex);
        String laserState_str = input.substring(secondCommaIndex + 1);

        // Convertir les chaînes en nombres
        long cibleX = X_str.toInt();
        long cibleY = Y_str.toInt();
        int laserState = laserState_str.toInt();

        // Limiter les coordonnées pour ne pas dépasser les pas minimaux et maximaux
        cibleX = constrain(cibleX, MIN_PAS_X, MAX_PAS_X);
        cibleY = constrain(cibleY, MIN_PAS_Y, MAX_PAS_Y);
        laserState = constrain(laserState, 0, 1); // État du laser doit être 0 ou 1

        // Afficher les coordonnées reçues et l'état du laser pour vérification
        Serial.print("Déplacement vers : X=");
        Serial.print(cibleX);
        Serial.print(" pas, Y=");
        Serial.print(cibleY);
        Serial.print(", Laser=");
        Serial.println(laserState);

        // Contrôler le relais (laser)
        if (laserState == 1) {
            digitalWrite(relayPin, HIGH); // Activer le relais (laser allumé)
        } else {
            digitalWrite(relayPin, LOW); // Désactiver le relais (laser éteint)
        }

        // Déplacer les moteurs vers les nouvelles coordonnées avec la vitesse normale
        deplacerAuxCoordonnees(cibleX, cibleY, VITESSE_NORMALE);
    } else {
        Serial.println("Erreur : veuillez entrer les coordonnées au format X,Y,Etat ou la commande 'init'.");
    }
}

// Si rien n'a été reçu depuis un certain temps, éteindre le laser
if (millis() - lastTime > TIMEOUT) {
    digitalWrite(relayPin, LOW); // Éteindre le laser
    Serial.println("Aucune commande reçue, laser éteint.");
}

void initialiserPosition() {
    // Initialisation pour l'axe X
    digitalWrite(M1dirpin, LOW); // Déplacement vers le capteur de fin de course
    while (digitalRead(limitSwitchX) == HIGH) {
        digitalWrite(M1steppin, LOW);
        delayMicroseconds(VITESSE_RALENTIE);
        digitalWrite(M1steppin, HIGH);
        delayMicroseconds(VITESSE_RALENTIE);
    }
    positionXActuelle = 0; // X est maintenant à 0

    // Initialisation pour l'axe Y

```



```

digitalWrite(M2dirpin, HIGH); // Déplacement vers le capteur de fin de course (inversé)
while (digitalRead(limitSwitchY) == HIGH) {
    digitalWrite(M2steppin, LOW);
    delayMicroseconds(VITESSE_RALENTIE);
    digitalWrite(M2steppin, HIGH);
    delayMicroseconds(VITESSE_RALENTIE);
}
positionYActuelle = 0; // Y est maintenant à 0
}

void deplacerAuxCoordonnees(long X_cible, long Y_cible, int vitesse) {
    long pasX = X_cible - positionXActuelle;
    long pasY = Y_cible - positionYActuelle;

    // Déterminer la direction de déplacement pour chaque moteur
    if (pasX > 0) {
        digitalWrite(M1dirpin, HIGH); // Aller vers l'avant pour X
    } else {
        digitalWrite(M1dirpin, LOW); // Aller vers l'arrière pour X
        pasX = -pasX; // Rendre positif pour la boucle
    }
    if (pasY > 0) {
        digitalWrite(M2dirpin, LOW); // Aller vers l'arrière pour Y (inversé)
    } else {
        digitalWrite(M2dirpin, HIGH); // Aller vers l'avant pour Y (inversé)
        pasY = -pasY; // Rendre positif pour la boucle
    }

    // Activation des moteurs
    digitalWrite(M1en, LOW); // Activer le moteur X
    digitalWrite(M2en, LOW); // Activer le moteur Y

    long totalSteps = max(pasX, pasY);

    // Boucle pour synchroniser les mouvements
    for (long step = 0; step < totalSteps; step++) {
        // Effectuer un pas sur X si besoin
        if (step < pasX) {
            digitalWrite(M1steppin, LOW);
            delayMicroseconds(vitesse);
            digitalWrite(M1steppin, HIGH);
            delayMicroseconds(vitesse);
        }

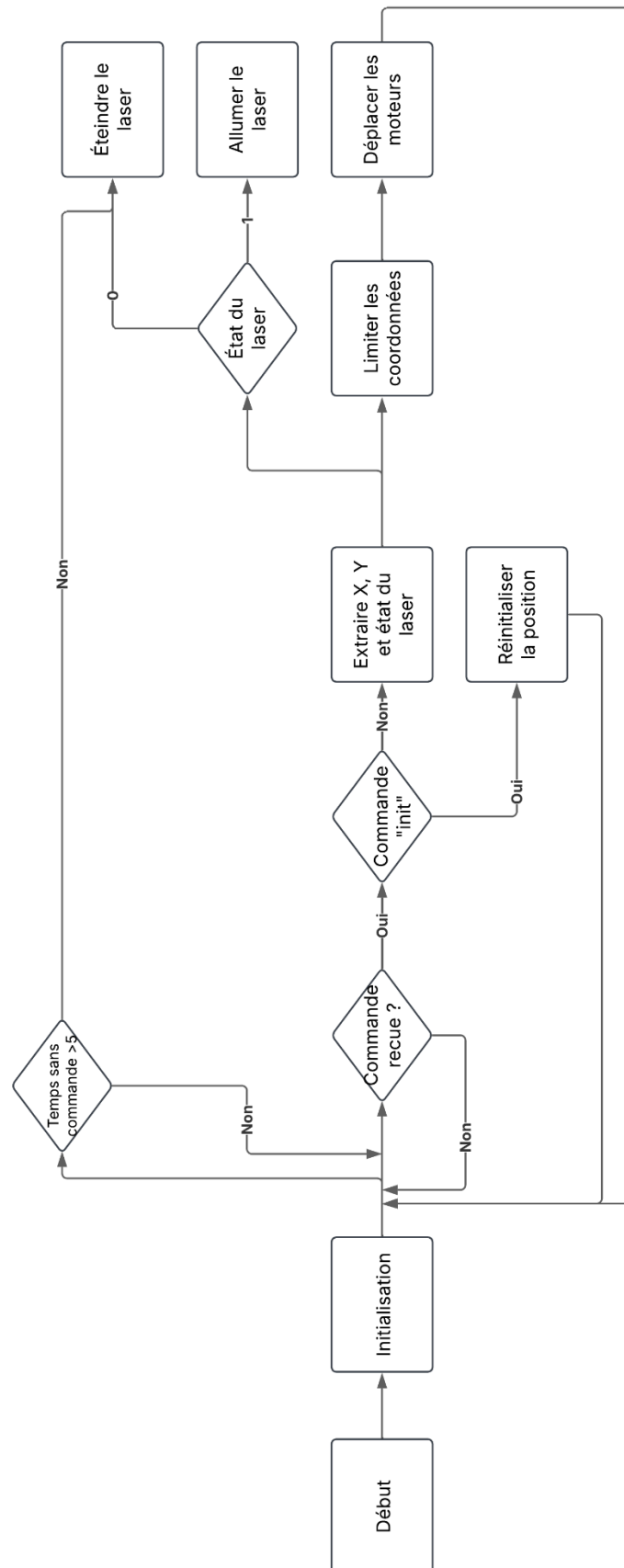
        // Effectuer un pas sur Y si besoin
        if (step < pasY) {
            digitalWrite(M2steppin, LOW);
            delayMicroseconds(vitesse);
            digitalWrite(M2steppin, HIGH);
            delayMicroseconds(vitesse);
        }
    }

    // Mettre à jour les positions actuelles
    positionXActuelle = X_cible;
    positionYActuelle = Y_cible;
}

```



C – Architecture Code Arduino



5. Développement durable / éthique/ santé et sécurité au travail

A - L'intégration de la thématique dans le Développement Durable

Notre projet aide à protéger l'environnement. Les frelons asiatiques sont un vrai danger pour les abeilles, qui sont essentielles à la nature car elles pollinisent les plantes. Avec notre système de détection utilisant l'intelligence artificielle, on peut repérer ces frelons plus rapidement et avec plus de précision. Cela permet d'intervenir uniquement là où c'est nécessaire, sans utiliser trop d'insecticides qui pourraient aussi nuire aux abeilles et aux autres insectes utiles. Grâce à cette technologie, on protège mieux la biodiversité tout en limitant l'impact négatif sur l'environnement.

Cependant, il faut aussi reconnaître que l'IA a un impact écologique, notamment à cause de la consommation d'énergie nécessaire à l'entraînement des modèles et au fonctionnement des systèmes de détection. L'objectif est donc de trouver un équilibre entre l'efficacité de notre solution et son empreinte environnementale.

B - L'intégration de la thématique dans la sécurité

Notre projet contribue aussi à la sécurité des personnes en permettant une identification rapide des frelons asiatiques dans différentes zones. Grâce à notre système, il est possible de repérer leur présence en temps réel et d'alerter les apiculteurs ou les professionnels concernés. Cela permet de mieux comprendre leur répartition et d'agir plus rapidement pour limiter leur impact. En réduisant le temps d'exposition des personnes aux frelons, notamment dans les zones à risque, notre solution participe à la prévention des piqûres et des accidents liés à ces insectes agressifs.

Cependant, l'utilisation d'un laser présente des risques pour les humains et les autres animaux. Afin de garantir un usage sécurisé, nous avons intégré des mesures de sécurité spécifiques. Un système de détection supplémentaire permet d'identifier la présence d'un humain dans la zone d'action du dispositif. Si une personne est détectée, le laser est immédiatement désactivé afin d'éviter tout risque d'exposition accidentelle. Cette précaution garantit que notre solution reste sécurisée tout en assurant son efficacité contre les frelons asiatiques.



Résumé

Notre projet, initié par Monsieur Michel Ousset, vise à développer un dispositif autonome de lutte contre les frelons asiatiques aux abords des ruches. L'objectif est de cibler ces nuisibles à l'aide d'un laser énergétique pour les effrayer ou les neutraliser. Pour cela, une caméra filme l'entrée de la ruche et un algorithme de détection basé sur YOLOv8 identifie les frelons et extrait leurs coordonnées. Nous avons conçu un prototype fonctionnel intégrant la détection et le suivi du frelon, permettant d'orienter un laser vers la cible grâce à un système de pointage motorisé piloté par Arduino. Malgré des défis techniques liés à l'utilisation du Jetson Nano, nous avons assemblé et testé l'ensemble du système. Les prochaines améliorations viseront à optimiser la rapidité de détection et de précision du modèle YOLO pour rendre le dispositif encore plus efficace.

Mots-clés : Intelligence artificielle – Laser – Frelon – Moteur pas à pas

Our project, initiated by Mr. Michel Ousset, aims to develop an autonomous system to combat Asian hornets near beehives. The goal is to target these pests using an energy laser to scare them away or neutralize them. To achieve this, a camera films the entrance of the hive, and a detection algorithm based on YOLOv8 identifies the hornets and extracts their coordinates. We have designed a fully functional prototype that integrates hornet detection and tracking, allowing the laser to be directed at the target through a motorized pointing system controlled by an Arduino. Despite technical challenges related to the use of the Jetson Nano, we have successfully assembled and tested the entire system. Future improvements will focus on optimizing the detection speed and the accuracy of the YOLO model to make the device even more effective.

Keywords : Artificial Intelligence – Laser – Hornet – Stepper Motor

